

LeadOS MVP Database Schema Document

1. Database Goal

LeadOS needs a simple database that supports the full demo journey:

Lead created → AI chat or voice conversation → Transcript saved → Scoring generated → Lead classified Hot/Warm/Cold → Hot assigned to RM → Warm gets WhatsApp follow up link → Admin dashboard updates

We are using **Supabase Postgres**, so the schema should be relational, easy to query, and simple enough to build quickly.

This schema is designed for one dummy client in the MVP, but every major table includes `client_id` so the product remains SaaS ready.

2. Core Tables

Final MVP tables:

clients
users
campaigns
leads
conversations
conversation_messages
lead_scores
rm_tasks
follow_ups
notifications
knowledge_documents
rm_feedback

Optional table if real RAG is implemented:

knowledge_chunks

Do not build extra CRM tables in MVP.

3. Enums

In Supabase, we can either create Postgres enums or use `text` fields with backend validation. For fastest MVP, use `text` fields and validate values in backend.

user_role

Allowed values:

admin
rm

Meaning:

admin: Client admin who manages leads, dashboards, RMs, settings
rm: Relationship Manager or sales user who handles assigned leads

lead_status

Allowed values:

new
pending_contact
conversation_completed
assigned_to_rm
follow_up_scheduled
converted
closed

Meaning:

new: Lead has just entered system
pending_contact: Lead is ready for AI chat or call
conversation_completed: AI conversation is completed
assigned_to_rm: Hot lead is assigned to RM
follow_up_scheduled: Warm lead has WhatsApp follow up ready or scheduled
converted: RM marked the lead as converted
closed: Cold, not interested, invalid, or no further action

lead_classification

Allowed values:

hot
warm
cold

Meaning:

hot: Strong intent and ready for RM
warm: Interested but needs follow up
cold: Low intent, weak fit, or not interested

conversation_channel

Allowed values:

chat
voice

conversation_status

Allowed values:

started
completed
failed

message_role

Allowed values:

user
assistant
system
tool

follow_up_channel

Allowed values:

whatsapp
email

For MVP, primary channel is whatsapp.

follow_up_status

Allowed values:

ready
opened
sent
cancelled

Meaning:

ready: Message and wa.me link generated
opened: RM/Admin opened WhatsApp link
sent: RM/Admin marked follow up as sent
cancelled: Follow up cancelled

rm_task_status

Allowed values:

```
pending
in_progress
converted
closed
```

objection_status

Allowed values inside JSON:

```
resolved
partially_resolved
unresolved
```

4. Table: clients

This represents an enterprise client workspace. For MVP, seed one dummy client.

Example dummy client:

```
FinPartner Pro
```

Fields

Field	Type	Required	Default	Meaning
id	uuid	yes	gen_random_uuid()	Primary key
name	text	yes	none	Client display name
slug	text	yes	none	Unique client slug
created_at	timestampz	yes	now()	Created time
updated_at	timestampz	yes	now()	Updated time

SQL

```
create table clients (
  id uuid primary key default gen_random_uuid(),
  name text not null,
  slug text unique not null,
  created_at timestampz not null default now(),
  updated_at timestampz not null default now()
);
```

Relations

```
clients 1 to many users
clients 1 to many campaigns
clients 1 to many leads
clients 1 to many conversations
clients 1 to many knowledge_documents
```

5. Table: users

This stores Admin and RM users. If using Supabase Auth, `id` can match `auth.users.id`.

Fields

Field	Type	Required	Default	Meaning
<code>id</code>	<code>uuid</code>	yes	<code>gen_random_uuid()</code>	User id
<code>client_id</code>	<code>uuid</code>	yes	<code>none</code>	Linked client
<code>name</code>	<code>text</code>	yes	<code>none</code>	User name
<code>email</code>	<code>text</code>	yes	<code>none</code>	Login email
<code>role</code>	<code>text</code>	yes	<code>none</code>	admin or rm
<code>avatar_url</code>	<code>text</code>	no	<code>null</code>	Profile image
<code>is_active</code>	<code>boolean</code>	yes	<code>true</code>	User active status
<code>created_at</code>	<code>timestampz</code>	yes	<code>now()</code>	Created time
<code>updated_at</code>	<code>timestampz</code>	yes	<code>now()</code>	Updated time

SQL

```
create table users (  
  id uuid primary key default gen_random_uuid(),  
  client_id uuid not null references clients(id) on delete cascade,  
  name text not null,  
  email text unique not null,  
  role text not null check (role in ('admin', 'rm')),  
  avatar_url text,  
  is_active boolean not null default true,  
  created_at timestampz not null default now(),  
  updated_at timestampz not null default now()  
);
```

Notes

For MVP roles:

```
admin can access Admin Portal  
rm can access RM Portal
```

6. Table: campaigns

Campaigns help group leads and show campaign performance.

Fields

Field	Type	Required	Default	Meaning
id	uuid	yes	gen_random_uuid()	Campaign id
client_id	uuid	yes	none	Linked client
name	text	yes	none	Campaign name
description	text	no	null	Campaign notes
source	text	no	null	website, csv, manual, demo
is_active	boolean	yes	true	Active campaign
created_at	timestampz	yes	now()	Created time
updated_at	timestampz	yes	now()	Updated time

SQL

```
create table campaigns (
  id uuid primary key default gen_random_uuid(),
  client_id uuid not null references clients(id) on delete cascade,
  name text not null,
  description text,
  source text,
  is_active boolean not null default true,
  created_at timestampz not null default now(),
  updated_at timestampz not null default now()
);
```

MVP Seed

Demo Partner Campaign

7. Table: leads

This is the central table. Every lead enters here.

Fields

Field	Type	Required	Default	Meaning
id	uuid	yes	gen_random_uuid()	Lead id
client_id	uuid	yes	none	Client workspace
campaign_id	uuid	no	null	Campaign source
assigned_rm_id	uuid	no	null	Assigned RM
name	text	yes	none	Lead name
phone	text	yes	none	Lead phone number
email	text	no	null	Lead email
city	text	no	null	Lead city

Field	Type	Required	Default	Meaning
role	text	no	null	advisor, mfd, insurance_agent, influencer, other
network_size	text	no	null	Approx network size
preferred_language	text	no	null	english, hindi, hinglish, etc
preferred_channel	text	no	null	chat or voice
whatsapp_consent	boolean	yes	false	Consent for WhatsApp follow up
source	text	no	null	website, csv, manual, crm
status	text	yes	new	Lead lifecycle status
classification	text	no	null	hot, warm, cold
latest_score	numeric	no	null	Latest total score
latest_summary	text	no	null	Latest AI summary
latest_next_action	text	no	null	Recommended next action
last_contacted_at	timestamp	no	null	Last AI or RM contact
converted_at	timestamp	no	null	Conversion time
closed_at	timestamp	no	null	Closed time
metadata	jsonb	yes	{}	Extra flexible data
created_at	timestamp	yes	now()	Created time
updated_at	timestamp	yes	now()	Updated time

SQL

```

create table leads (
  id uuid primary key default gen_random_uuid(),
  client_id uuid not null references clients(id) on delete cascade,
  campaign_id uuid references campaigns(id) on delete set null,
  assigned_rm_id uuid references users(id) on delete set null,

  name text not null,
  phone text not null,
  email text,
  city text,
  role text,
  network_size text,

  preferred_language text,
  preferred_channel text check (preferred_channel is null or
  preferred_channel in ('chat', 'voice')),
  whatsapp_consent boolean not null default false,

  source text,
  status text not null default 'new' check (
    status in (
      'new',
      'pending_contact',
      'conversation_completed',

```

```

        'assigned_to_rm',
        'follow_up_scheduled',
        'converted',
        'closed'
    )
),
classification text check (
    classification is null or classification in ('hot', 'warm', 'cold')
),

latest_score numeric,
latest_summary text,
latest_next_action text,
last_contacted_at timestamptz,
converted_at timestamptz,
closed_at timestamptz,

metadata jsonb not null default '{} '::jsonb,

created_at timestamptz not null default now(),
updated_at timestamptz not null default now()
);

```

Indexes

```

create index leads_client_id_idx on leads(client_id);
create index leads_campaign_id_idx on leads(campaign_id);
create index leads_assigned_rm_id_idx on leads(assigned_rm_id);
create index leads_status_idx on leads(status);
create index leads_classification_idx on leads(classification);
create index leads_created_at_idx on leads(created_at);

```

Relations

```

lead belongs to client
lead may belong to campaign
lead may be assigned to RM
lead has many conversations
lead has many scores
lead may have RM task
lead may have follow up

```

8. Table: conversations

Stores one chat or voice session.

Fields

Field	Type	Required	Default	Meaning
id	uuid	yes	gen_random_uuid()	Conversation id
client_id	uuid	yes	none	Client workspace

Field	Type	Required	Default	Meaning
lead_id	uuid	yes	none	Lead
channel	text	yes	none	chat or voice
status	text	yes	started	started, completed, failed
provider	text	no	null	vapi, web_chat
provider_conversation_id	text	no	null	Vapi call id or chat session id
language	text	no	null	Conversation language
started_at	timestampz	yes	now()	Start time
ended_at	timestampz	no	null	End time
duration_seconds	integer	no	null	Duration
transcript_text	text	no	null	Full transcript
summary	text	no	null	AI summary
raw_provider_payload	jsonb	yes	{}	Vapi or provider raw data
metadata	jsonb	yes	{}	Extra data
created_at	timestampz	yes	now()	Created time
updated_at	timestampz	yes	now()	Updated time

SQL

```

create table conversations (
  id uuid primary key default gen_random_uuid(),
  client_id uuid not null references clients(id) on delete cascade,
  lead_id uuid not null references leads(id) on delete cascade,

  channel text not null check (channel in ('chat', 'voice')),
  status text not null default 'started' check (status in ('started',
'completed', 'failed')),

  provider text,
  provider_conversation_id text,
  language text,

  started_at timestampz not null default now(),
  ended_at timestampz,
  duration_seconds integer,

  transcript_text text,
  summary text,

  raw_provider_payload jsonb not null default '{} '::jsonb,
  metadata jsonb not null default '{} '::jsonb,

  created_at timestampz not null default now(),
  updated_at timestampz not null default now()
);

```

Indexes

```
create index conversations_client_id_idx on conversations(client_id);
create index conversations_lead_id_idx on conversations(lead_id);
create index conversations_channel_idx on conversations(channel);
create index conversations_status_idx on conversations(status);
```

9. Table: conversation_messages

Stores each message or transcript turn.

Fields

Field	Type	Required	Default	Meaning
id	uuid	yes	gen_random_uuid()	Message id
client_id	uuid	yes	none	Client workspace
conversation_id	uuid	yes	none	Conversation
lead_id	uuid	yes	none	Lead
role	text	yes	none	user, assistant, system, tool
content	text	yes	none	Message text
language	text	no	null	Message language
sequence_no	integer	yes	none	Message order
timestamp	timestampz	yes	now()	Message time
tool_name	text	no	null	Tool name if role is tool
tool_payload	jsonb	no	null	Tool result or args
metadata	jsonb	yes	{}	Extra data
created_at	timestampz	yes	now()	Created time

SQL

```
create table conversation_messages (
  id uuid primary key default gen_random_uuid(),
  client_id uuid not null references clients(id) on delete cascade,
  conversation_id uuid not null references conversations(id) on delete
cascade,
  lead_id uuid not null references leads(id) on delete cascade,

  role text not null check (role in ('user', 'assistant', 'system', 'tool')),
  content text not null,
  language text,

  sequence_no integer not null,
  timestamp timestampz not null default now(),

  tool_name text,
  tool_payload jsonb,
```

```

    metadata jsonb not null default '{}'::jsonb,

    created_at timestampz not null default now()
);

```

Indexes

```

create index conversation_messages_conversation_id_idx on
conversation_messages(conversation_id);
create index conversation_messages_lead_id_idx on
conversation_messages(lead_id);
create index conversation_messages_sequence_idx on
conversation_messages(conversation_id, sequence_no);

```

10. Table: lead_scores

Stores scoring LLM output and explanation.

Fields

Field	Type	Required	Default	Meaning
id	uuid	yes	gen_random_uuid()	Score id
client_id	uuid	yes	none	Client workspace
lead_id	uuid	yes	none	Lead
conversation_id	uuid	yes	none	Conversation
classification	text	yes	none	hot, warm, cold
total_score	numeric	yes	none	Total score
readiness_score	numeric	yes	none	Out of 40
engagement_score	numeric	yes	none	Out of 35
fit_score	numeric	yes	none	Out of 25
reason	text	yes	none	Human readable reason
recommended_next_action	text	no	null	RM or system next step
positive_signals	jsonb	yes	[]	Reasons in favor
negative_signals	jsonb	yes	[]	Risks or weak signals
objections	jsonb	yes	[]	Objections extracted
extracted_data	jsonb	yes	{}	Structured extraction
raw_llm_output	jsonb	yes	{}	Original model response
created_at	timestampz	yes	now()	Created time

SQL

```

create table lead_scores (
    id uuid primary key default gen_random_uuid(),
    client_id uuid not null references clients(id) on delete cascade,

```

```

    lead_id uuid not null references leads(id) on delete cascade,
    conversation_id uuid not null references conversations(id) on delete
cascade,

    classification text not null check (classification in ('hot', 'warm',
'cold')),
    total_score numeric not null,

    readiness_score numeric not null,
    engagement_score numeric not null,
    fit_score numeric not null,

    reason text not null,
    recommended_next_action text,

    positive_signals jsonb not null default '[]'::jsonb,
    negative_signals jsonb not null default '[]'::jsonb,
    objections jsonb not null default '[]'::jsonb,
    extracted_data jsonb not null default '{}'::jsonb,
    raw_llm_output jsonb not null default '{}'::jsonb,

    created_at timestamptz not null default now()
);

```

Indexes

```

create index lead_scores_client_id_idx on lead_scores(client_id);
create index lead_scores_lead_id_idx on lead_scores(lead_id);
create index lead_scores_conversation_id_idx on lead_scores(conversation_id);
create index lead_scores_classification_idx on lead_scores(classification);
create index lead_scores_created_at_idx on lead_scores(created_at);

```

Example objections JSON

```

[
  {
    "type": "existing_broker",
    "lead_statement": "I already work with another broker",
    "status": "resolved",
    "ai_response_summary": "Compared 100 percent brokerage and daily payouts
against current broker model"
  }
]

```

Example extracted_data JSON

```

{
  "language": "hinglish",
  "role": "financial_advisor",
  "network_size": "25 to 30",
  "current_broker_status": true,
  "whatsapp_consent": true,
  "callback_time": null,

```

```

    "buying_intent": "asked_for_onboarding_details"
}

```

11. Table: rm_tasks

Creates a work item for RM when a lead is Hot.

Fields

Field	Type	Required	Default	Meaning
id	uuid	yes	gen_random_uuid()	Task id
client_id	uuid	yes	none	Client workspace
lead_id	uuid	yes	none	Lead
rm_id	uuid	yes	none	Assigned RM
conversation_id	uuid	no	null	Related conversation
lead_score_id	uuid	no	null	Related score
title	text	yes	none	Task title
status	text	yes	pending	pending, in_progress, converted, closed
priority	text	yes	high	high, medium, low
recommended_action	text	no	null	What RM should do
suggested_opening_line	text	no	null	Suggested first line
due_at	timestampz	no	null	Due time
completed_at	timestampz	no	null	Completed time
created_at	timestampz	yes	now()	Created time
updated_at	timestampz	yes	now()	Updated time

SQL

```

create table rm_tasks (
  id uuid primary key default gen_random_uuid(),
  client_id uuid not null references clients(id) on delete cascade,
  lead_id uuid not null references leads(id) on delete cascade,
  rm_id uuid not null references users(id) on delete cascade,
  conversation_id uuid references conversations(id) on delete set null,
  lead_score_id uuid references lead_scores(id) on delete set null,

  title text not null,
  status text not null default 'pending' check (status in ('pending',
'in_progress', 'converted', 'closed')),
  priority text not null default 'high' check (priority in ('high', 'medium',
'low')),

  recommended_action text,
  suggested_opening_line text,

```

```

    due_at timestamptz,
    completed_at timestamptz,

    created_at timestamptz not null default now(),
    updated_at timestamptz not null default now()
);

```

Indexes

```

create index rm_tasks_client_id_idx on rm_tasks(client_id);
create index rm_tasks_rm_id_idx on rm_tasks(rm_id);
create index rm_tasks_lead_id_idx on rm_tasks(lead_id);
create index rm_tasks_status_idx on rm_tasks(status);

```

12. Table: follow_ups

Stores WhatsApp follow up link for Warm leads.

Fields

Field	Type	Required	Default	Meaning
id	uuid	yes	gen_random_uuid()	Follow up id
client_id	uuid	yes	none	Client workspace
lead_id	uuid	yes	none	Lead
rm_id	uuid	no	null	RM if assigned
conversation_id	uuid	no	null	Related conversation
channel	text	yes	whatsapp	whatsapp or email
status	text	yes	ready	ready, opened, sent, cancelled
message	text	yes	none	Follow up message
wa_me_link	text	no	null	WhatsApp deep link
scheduled_at	timestamptz	no	null	Scheduled time
opened_at	timestamptz	no	null	Link opened time
sent_at	timestamptz	no	null	Marked sent time
created_at	timestamptz	yes	now()	Created time
updated_at	timestamptz	yes	now()	Updated time

SQL

```

create table follow_ups (
    id uuid primary key default gen_random_uuid(),
    client_id uuid not null references clients(id) on delete cascade,
    lead_id uuid not null references leads(id) on delete cascade,
    rm_id uuid references users(id) on delete set null,
    conversation_id uuid references conversations(id) on delete set null,

```

```

    channel text not null default 'whatsapp' check (channel in ('whatsapp',
'email')),
    status text not null default 'ready' check (status in ('ready', 'opened',
'sent', 'cancelled')),

    message text not null,
    wa_me_link text,

    scheduled_at timestamptz,
    opened_at timestamptz,
    sent_at timestamptz,

    created_at timestamptz not null default now(),
    updated_at timestamptz not null default now()
);

```

Indexes

```

create index follow_ups_client_id_idx on follow_ups(client_id);
create index follow_ups_lead_id_idx on follow_ups(lead_id);
create index follow_ups_status_idx on follow_ups(status);
create index follow_ups_scheduled_at_idx on follow_ups(scheduled_at);

```

13. Table: notifications

For in app notifications in Admin or RM portals.

Fields

Field	Type	Required	Default	Meaning
id	uuid	yes	gen_random_uuid()	Notification id
client_id	uuid	yes	none	Client workspace
user_id	uuid	yes	none	Receiver
title	text	yes	none	Notification title
message	text	yes	none	Notification body
type	text	no	null	hot_lead_assigned, follow_up_due, score_ready
is_read	boolean	yes	false	Read status
entity_type	text	no	null	lead, task, follow_up
entity_id	uuid	no	null	Related entity id
created_at	timestamptz	yes	now()	Created time
read_at	timestamptz	no	null	Read time

SQL

```

create table notifications (
  id uuid primary key default gen_random_uuid(),
  client_id uuid not null references clients(id) on delete cascade,
  user_id uuid not null references users(id) on delete cascade,

  title text not null,
  message text not null,
  type text,
  is_read boolean not null default false,

  entity_type text,
  entity_id uuid,

  created_at timestamptz not null default now(),
  read_at timestamptz
);

```

Indexes

```

create index notifications_client_id_idx on notifications(client_id);
create index notifications_user_id_idx on notifications(user_id);
create index notifications_is_read_idx on notifications(is_read);

```

14. Table: knowledge_documents

Stores static script, FAQ, onboarding info, objection guide.

Fields

Field	Type	Required	Default	Meaning
id	uuid	yes	gen_random_uuid()	Document id
client_id	uuid	yes	none	Client workspace
title	text	yes	none	Document title
document_type	text	yes	none	script, faq, onboarding, objection_guide
content	text	yes	none	Full document content
is_active	boolean	yes	true	Used by AI or not
created_at	timestamptz	yes	now()	Created time
updated_at	timestamptz	yes	now()	Updated time

SQL

```

create table knowledge_documents (
  id uuid primary key default gen_random_uuid(),
  client_id uuid not null references clients(id) on delete cascade,

  title text not null,
  document_type text not null check (

```

```

        document_type in ('script', 'faq', 'onboarding', 'objection_guide')
    ),
    content text not null,
    is_active boolean not null default true,

    created_at timestamptz not null default now(),
    updated_at timestamptz not null default now()
);

```

Indexes

```

create index knowledge_documents_client_id_idx on
knowledge_documents(client_id);
create index knowledge_documents_type_idx on
knowledge_documents(document_type);

```

15. Optional Table: knowledge_chunks

Only create this if real RAG with vector search is implemented.

If time is tight, skip this and load active `knowledge_documents` directly into prompt or simple text search.

Fields

Field	Type	Required	Default	Meaning
id	uuid	yes	gen_random_uuid()	Chunk id
client_id	uuid	yes	none	Client workspace
document_id	uuid	yes	none	Parent document
chunk_text	text	yes	none	Chunk content
chunk_index	integer	yes	none	Order
embedding	vector	no	null	Vector embedding if pgvector enabled
metadata	jsonb	yes	{}	Extra data
created_at	timestamptz	yes	now()	Created time

SQL

```

create table knowledge_chunks (
    id uuid primary key default gen_random_uuid(),
    client_id uuid not null references clients(id) on delete cascade,
    document_id uuid not null references knowledge_documents(id) on delete
    cascade,

    chunk_text text not null,
    chunk_index integer not null,
    embedding vector,
    metadata jsonb not null default '{}'::jsonb,

```

```

    created_at timestampz not null default now()
);

```

Note

This requires pgvector:

```
create extension if not exists vector;
```

For 4 day MVP, this table is optional.

16. Table: rm_feedback

Stores RM corrections and feedback for scoring improvement.

Fields

Field	Type	Required	Default	Meaning
id	uuid	yes	gen_random_uuid()	Feedback id
client_id	uuid	yes	none	Client workspace
lead_id	uuid	yes	none	Lead
rm_id	uuid	yes	none	RM
lead_score_id	uuid	no	null	Related score
original_classification	text	no	null	AI classification
corrected_classification	text	no	null	RM corrected classification
original_score	numeric	no	null	AI score
corrected_score	numeric	no	null	RM corrected score
feedback_type	text	no	null	score_wrong, objection_wrong, bad_response, good_response
comment	text	no	null	RM comment
created_at	timestampz	yes	now()	Created time

SQL

```

create table rm_feedback (
    id uuid primary key default gen_random_uuid(),
    client_id uuid not null references clients(id) on delete cascade,
    lead_id uuid not null references leads(id) on delete cascade,
    rm_id uuid not null references users(id) on delete cascade,
    lead_score_id uuid references lead_scores(id) on delete set null,

    original_classification text check (

```

```

        original_classification is null or original_classification in ('hot',
'warm', 'cold')
    ),
    corrected_classification text check (
        corrected_classification is null or corrected_classification in ('hot',
'warm', 'cold')
    ),

    original_score numeric,
    corrected_score numeric,

    feedback_type text,
    comment text,

    created_at timestamptz not null default now()
);

```

Indexes

```

create index rm_feedback_client_id_idx on rm_feedback(client_id);
create index rm_feedback_lead_id_idx on rm_feedback(lead_id);
create index rm_feedback_rm_id_idx on rm_feedback(rm_id);

```

17. Relationships Summary

```

clients
  → users
  → campaigns
  → leads
    → conversations
      → conversation_messages
    → lead_scores
    → rm_tasks
    → follow_ups
    → rm_feedback
  → notifications
  → knowledge_documents
    → knowledge_chunks optional

```

Detailed relationship map:

```

clients.id → users.client_id
clients.id → campaigns.client_id
clients.id → leads.client_id
campaigns.id → leads.campaign_id
users.id → leads.assigned_rm_id
leads.id → conversations.lead_id
conversations.id → conversation_messages.conversation_id
leads.id → lead_scores.lead_id
conversations.id → lead_scores.conversation_id
leads.id → rm_tasks.lead_id
users.id → rm_tasks.rm_id
leads.id → follow_ups.lead_id

```

users.id → notifications.user_id
leads.id → rm_feedback.lead_id

18. Routing Rules

Hot lead

When score classification is hot:

```
Insert lead_scores row
Update leads.classification = hot
Update leads.latest_score
Update leads.latest_summary
Update leads.latest_next_action
Update leads.status = assigned_to_rm
Assign RM if not assigned
Create rm_tasks row
Create notification for RM
```

Warm lead

When score classification is warm:

```
Insert lead_scores row
Update leads.classification = warm
Update leads.latest_score
Update leads.latest_summary
Update leads.latest_next_action
Update leads.status = follow_up_scheduled
Generate WhatsApp message
Generate wa.me link
Create follow_ups row
```

Cold lead

When score classification is cold:

```
Insert lead_scores row
Update leads.classification = cold
Update leads.latest_score
Update leads.latest_summary
Update leads.latest_next_action
Update leads.status = closed
No RM task created
```

19. Portal Data Requirements

Admin Dashboard Queries

Admin needs:

Total leads
Leads by status
Hot/Warm/Cold count
Language distribution
Campaign performance
Recent conversations
Objection trends from latest score JSON
Follow up count
Converted count

Can be calculated from:

leads
conversations
lead_scores
follow_ups
rm_tasks

RM Portal Queries

RM needs:

Assigned Hot leads
RM tasks
Lead detail
Transcript messages
Score reason
Objections
Suggested action
Notifications

Can be calculated from:

leads
rm_tasks
conversations
conversation_messages
lead_scores
notifications
follow_ups

Lead Detail Page Needs

Use:

leads
latest conversation
conversation_messages
latest lead_scores
rm_tasks if Hot
follow_ups if Warm
rm_feedback if available

20. Recommended Build Order

Build tables in this order:

1. clients
2. users
3. campaigns
4. leads
5. conversations
6. conversation_messages
7. lead_scores
8. rm_tasks
9. follow_ups
10. notifications
11. knowledge_documents
12. rm_feedback
13. knowledge_chunks optional

This order lets backend and frontend start quickly.

21. MVP Seed Data

Seed one client:

Client: FinPartner Pro
Campaign: Demo Partner Campaign
Admin: admin@lead-os.demo
RM: rm@lead-os.demo

Seed knowledge documents:

Demo sales script
Demo FAQ
Demo onboarding guide
Demo objection guide

Seed example leads if needed:

Amit Sharma, Hinglish, Financial Advisor
Priya Mehta, English, MFD
Rahul Verma, Hindi, Insurance Agent

22. Final MVP Schema Decision

Build:

clients
users
campaigns
leads

conversations
conversation_messages
lead_scores
rm_tasks
follow_ups
notifications
knowledge_documents
rm_feedback

Optional:

knowledge_chunks

Do not build:

billing
subscriptions
tenant themes
full audit logs
provider configs
workflow run tables
separate objections table
separate score factors table
campaign metrics table
RM metrics table

Analytics can be calculated from existing tables for MVP.

23. Why This Schema Is Enough

This schema supports:

Lead capture
AI chat
AI voice
Transcript storage
Post conversation scoring
Hot/Warm/Cold routing
RM handoff
WhatsApp wa.me follow up
Dashboard analytics
RM notifications
Basic RAG documents
RM feedback